

Mordisbot

*Sistema de recomendación gastronómica
con procesamiento de eventos en tiempo real*

White Paper — Trabajo Práctico Integrador Final

Ingeniería de Software II · 1er cuatrimestre 2026

Equipo Tiziano Fuchinecco — Legajo 64191
Matías Romanato — Legajo 62072

Profesor Emilio Rasic

Eje Técnico Kafka — sistemas distribuidos, pub/sub
Implementado vía Redpanda

Fecha 18 de junio de 2026

Índice

1. Problema y producto

2. Eje Funcional

- 2.1 Por qué un sistema reactivo a eventos
- 2.2 Ejemplos de utilidad para el negocio
- 2.3 Comparativa de alternativas

3. Eje Técnico

- 3.1 El modelo de Kafka
- 3.2 Mapeo a Mordisbot
- 3.3 Prueba de concepto: Tendencia detectada

4. Riesgos y próximos pasos

Anexos

1. Problema y producto

Problema	Mordisbot
Filtros rígidos (Google Maps, TripAdvisor) Datos desactualizados No considera presupuesto, ambiente, momento Restaurantes chicos invisibles sin publicidad cara	Lenguaje natural vía Telegram Datos cargados por los propios dueños Pondera zona, presupuesto, cocina, ambiente, momento Canal accesible para restaurantes de cualquier tamaño

Modelo de negocio

Fuente de ingreso	Descripción	Sostenido por
1. Prioridad paga	Restaurantes pagan para aparecer más. Techo configurable para no degradar calidad.	Pipeline de recomendación (ranker β)
2. Tendencias B2B	Datos agregados/anonimizados: qué se busca, dónde y cuándo. Se venden a cadenas e inversores.	Bus de eventos + CEP (esta PoC)

Ambas fuentes de ingreso dependen de la **arquitectura de eventos**: la prioridad paga necesita el ranker en el pipeline, y las tendencias B2B son el subproducto directo del bus pub/sub — se calculan sobre el tráfico que ya existe, sin costo adicional de infraestructura.

2. Eje Funcional

2.1 Pipeline Pipes & Filters



Los eventos se publican al **bus pub/sub** en los puntos de salida relevantes (tras NLU, tras el ranker, tras la entrega) sin bloquear el camino crítico.

2.2 Por qué eventos — monolito vs. reactivo

	Sistema sincrónico / monolítico	Sistema reactivo a eventos
Agregar analítica	Tocar el camino crítico de respuesta al usuario	Nuevo consumidor del bus, sin tocar el pipeline
Caída de un módulo	Puede romper toda la respuesta	El pipeline sigue; los eventos se acumulan y reprocesa
Escalar un componente	Reescalar todo junto	Réplicas independientes por consumidor
Detectar tendencias	Reporte manual o batch nocturno	CEP sobre el stream en tiempo real

2.3 Reglas CEP — valor de negocio

Regla CEP	Patrón (ventana)	Acción de negocio
1. Tendencia detectada (PoC)	≥ 50 búsquedas cocina X en zona Y en 72 h	Señal al producto B2B + influencers afines
2. Alerta de coste	Gasto IA proyectado > presupuesto diario (1 h)	Switch a modelo chico + respuestas plantilla
3. Calidad cae	≥ 5 NPS < 5 del mismo restaurante en 24 h	Pausar prioridad paga + notificar al dueño
4. Restaurante caliente	≥ 10 visitas/reservas mismo local en 24 h	Marcar tendencia B2B + aumentar peso reputación

Clave: las decisiones más valiosas surgen de la **correlación de eventos en el tiempo**, no de uno solo. ESP agrega sin actuar; CEP correlaciona y dispara la acción.

2.4 Comparativa de alternativas

	Kafka - elegido	RabbitMQ	Redpanda - implementación
Modelo	Log distribuido, retención configurable	Cola con routing flexible (exchanges)	Mismo protocolo Kafka, un solo container
Fan-out / replay	Consumer groups independientes + replay	Requiere cola por consumidor, sin replay	Idem Kafka
Ventanas largas (CEP)	Log retenido hasta 7 d (o más)	Cola descarta tras el consumo	Idem Kafka
Operación	Requiere KRaft / Zookeeper en prod	Simple, más fácil de operar	Un binario (sin Zookeeper), sin Zookeeper
Decisión	Modelo conceptual y de diseño	<i>No elegido</i>	Motor de la PoC

3. Eje Técnico

3.1 El modelo de Kafka

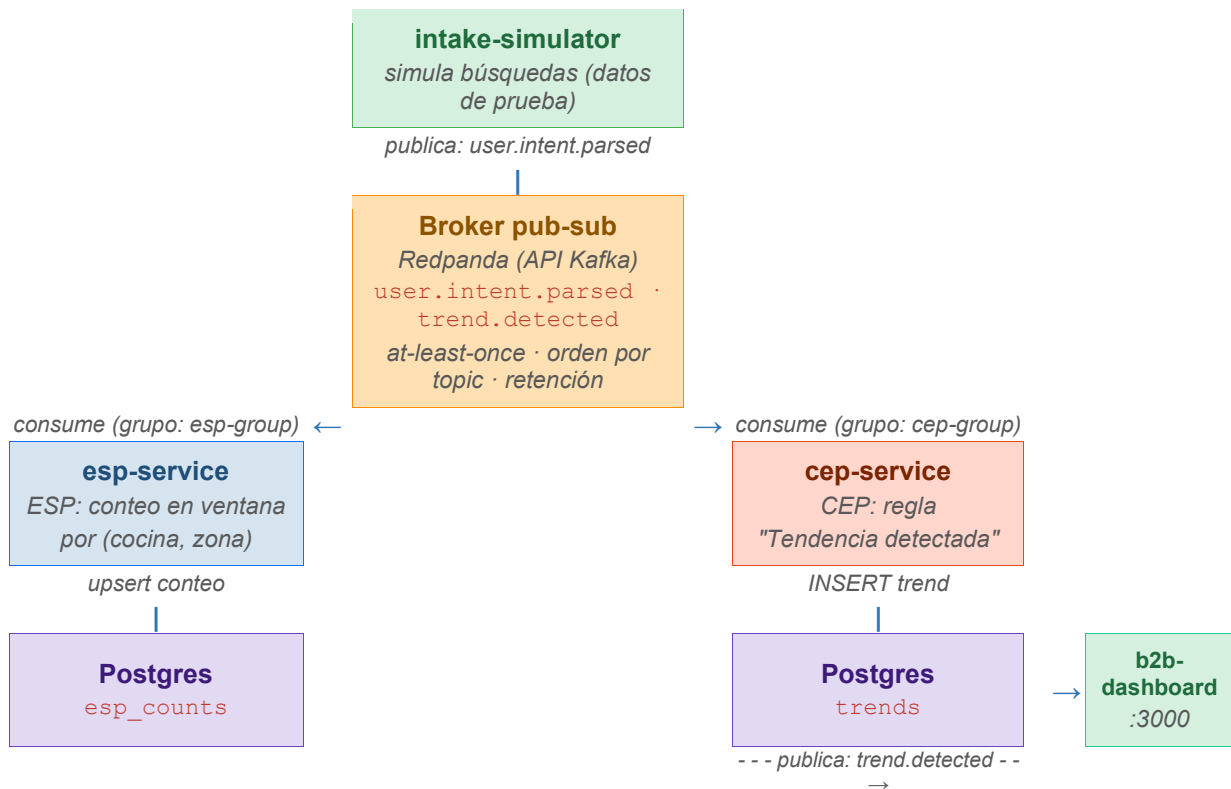
Concepto	Qué es	Para Mordisbot
Topic / Partición	Flujo nombrado de eventos dividido en particiones (logs físicos). Orden garantizado dentro de la partición.	Partición por <code>chat_id</code> → orden por conversación; por <code>rest_id</code> → actividad por restaurante
Offset	Posición secuencial e inmutable. Kafka no borra al leer; cada consumer lleva el suyo.	Permite replay: reconstruir métricas o depurar desde cualquier punto de la historia
Consumer group	Conjunto de consumers que se reparten las particiones. Grupos distintos = fan-out.	<code>esp-group</code> y <code>cep-group</code> leen el mismo topic en paralelo, sin pisarse
At-least-once	Un mensaje puede entregarse más de una vez ante reintentos.	Consumers idempotentes: upsert en lugar de insert ciego
Retención / Replay	Los mensajes se retienen por tiempo configurable (ej. 7 días).	Un consumer nuevo puede procesar toda la historia; clave para las reglas CEP de ventanas largas

3.2 Mapeo a Mordisbot — topics y partition keys

Topic	Partition key	Productor → Consumidores	PoC
<code>user.message.in</code>	<code>chat_id</code>	Adapter Telegram → Orquestador, Logger	—
<code>user.intent.parsed</code>	<code>chat_id</code>	NLU → Analítica, CEP, Logger	implementado
<code>reco.proposed</code>	<code>chat_id</code>	Ranker → Validador, Analítica	—
<code>reco.delivered</code>	<code>chat_id</code>	Composer → CEP, Analítica	—
<code>feedback.received</code>	<code>rest_id</code>	Bot NPS → CEP, Analítica, Ranker	—
<code>restaurant.update</code>	<code>rest_id</code>	Onboarding → Indexador Vector DB	—
<code>trend.detected</code>	—	cep-service → (B2B, influencers)	implementado

3.3 Prueba de concepto: Tendencia detectada

Comando: `docker compose up -d --build + docker compose run --rm intake-simulator`



Regla real vs. demo: ≥ 50 búsquedas de cocina X en zona Y en 72 h $\rightarrow$ demo: ≥ 5 en 60 s (mismo patrón, escala observable en vivo). Parámetros configurables vía variables de entorno.

Resultado verificado

Servicio	Observación verificada	Valor
esp-service	sushi/Palermo count = 1, 2, 3, 4, 5, 6 (6 eventos en <30s)	Agrega todos los eventos sin condición (ESP)
cep-service	TENDENCIA DETECTADA al 5º evento $\rightarrow$ 1 INSERT en trends	Dispara exactamente una vez al cruzar el umbral (CEP)
Otras combinaciones	pizza/Belgrano, parrilla/Caballito $\rightarrow$ sin tendencia	La regla es selectiva, no genera falsos positivos
b2b-dashboard	Muestra sushi Palermo 5 60s. Auto-refresh 5s.	El producto B2B, tangible y en tiempo real

Diferencia clave: ESP vs. CEP

	ESP (esp_counts)	CEP (trends)
Cuenta hasta	6 (todos los eventos)	5 (momento del cruce)
¿Sigue después del umbral?	Sí, cuenta todos	No, dispara una vez (Set notified)
Propósito	Dashboard / métricas continuas	Acción de negocio — no se repite por el mismo pico

4. Riesgos y próximos pasos

Riesgos identificados y mitigaciones

Riesgo	Mitigación	Regla/mecanismo
Latencia por 2 llamadas LLM	Validador más chico + 1 reintento máximo	Filtro 6 usa Haiku vs Sonnet en filtro 3
Costo de IA fuera de control	Switch automático a modelo chico + plantillas	Regla CEP 2. Alerta de coste
Prioridad paga degrada calidad	Techo configurable en β del ranker	Regla CEP 3. pausa boost si NPS cae
Datos inconsistentes de restaurantes	Validación de esquema antes de embeddear	Indexador rechaza + evento de error al dueño
Complejidad operativa de Kafka en prod	Redpanda Cloud gestionado reduce la operación	Trade-off resuelto en la PoC con Railway

Deploy implementado

La PoC corre en dos entornos complementarios:

Entorno	Stack	Acceso
Local (demo completa)	Docker Compose: Redpanda + Postgres + 4 servicios Node	<code>docker compose up -d --build</code>
Nube (endpoint B2B)	Railway: Postgres gestionado + b2b-dashboard (Dockerfile)	<code>GET https://mordisbot-production.up.railway.app/api/trends</code>

El entorno de nube expone el endpoint B2B (`/api/trends`) con datos de ejemplo que simulan el resultado del flujo Kafka. El flujo completo con broker real se demuestra en el entorno local. Esta separación refleja una decisión práctica: Kafka requiere procesos de larga duración que no encajan en plataformas serverless; para producción real la opción es Redpanda Cloud (gestionado) o una VM dedicada.

Próximos pasos

Próximo paso	Impacto
Implementar filtros NLU, RAG, ranker, validador reales	Pipeline completo sin intake-simulator
Consumidores reales de trend.detected (B2B, influencers)	Activar la 2da fuente de ingresos en producción
Más reglas CEP declarativas (preferencia personal, restaurante caliente)	Sin tocar el pipeline existente
Deploy del flujo Kafka completo (Redpanda Cloud o VM dedicada)	Eliminar el intake-simulator del entorno de nube

Anexos

Anexo A. Diagrama de la arquitectura completa (Pipes & Filters 8 filtros + bus pub/sub + ESP/CEP) — Actividad 1.

Anexo B. Diagrama SVG de la PoC — docs/poc-architecture.svg en el repositorio.

Anexo C. Schemas completos de topics (user.intent.parsed, trend.detected) y tablas Postgres (esp_counts, trends) — docs/02-ARQUITECTURA-POC.md.

Anexo D. Stack tecnológico completo con fila agregada: Broker pub/sub → Kafka (modelo) / Redpanda (implementación).

Anexo E. Wireframe del flujo conversacional en Telegram — caso de negocio.